

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	KAVALI BALAJI	Reg.No.:	192525402
Department:	AI&ML	Date:	03-09-2026

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I KAVALI BALAJI(192525402)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. DMA-BASED DATA TRANSFER ALGORITHM

Question: Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Aim

To design a DMA-based data transfer algorithm that transfers a block of data from an I/O device directly to main memory, uses interrupts to notify the CPU, and minimizes continuous CPU involvement.

Problem Understanding

- Input: I/O device, starting memory address, transfer size, and DMA controller.
- Output: Data block stored correctly in memory and completion status reported through an interrupt.
- Constraint: CPU should configure the transfer but should not copy every data item.

Algorithm

- Initialize the DMA controller.
- Configure the source I/O device, destination memory address, transfer size, and transfer direction.
- Start the DMA transfer.
- Allow DMA to transfer data directly between the I/O device and memory.
- CPU continues other work while the transfer is in progress.
- When the transfer completes, DMA raises an interrupt.
- CPU executes the DMA interrupt service routine and verifies completion.

Pseudocode

```
ALGORITHM DMA_Transfer
INPUT: IO_Device, Memory_Start, Transfer_Size
OUTPUT: Transfer_Status

BEGIN
    Initialize DMA_Controller
    Set DMA.Source ← IO_Device
    Set DMA.Destination ← Memory_Start
    Set DMA.Count ← Transfer_Size
    Set DMA.Direction ← IO_TO_MEMORY
    Enable DMA_Interrupt

    Start DMA_Controller

    WHILE DMA.Status = BUSY DO
        CPU performs other useful tasks
    END WHILE

    ON DMA_Interrupt DO
        IF DMA.Status = TRANSFER_COMPLETE THEN
            Transfer_Status ← "SUCCESS"
            Clear DMA_Interrupt
```

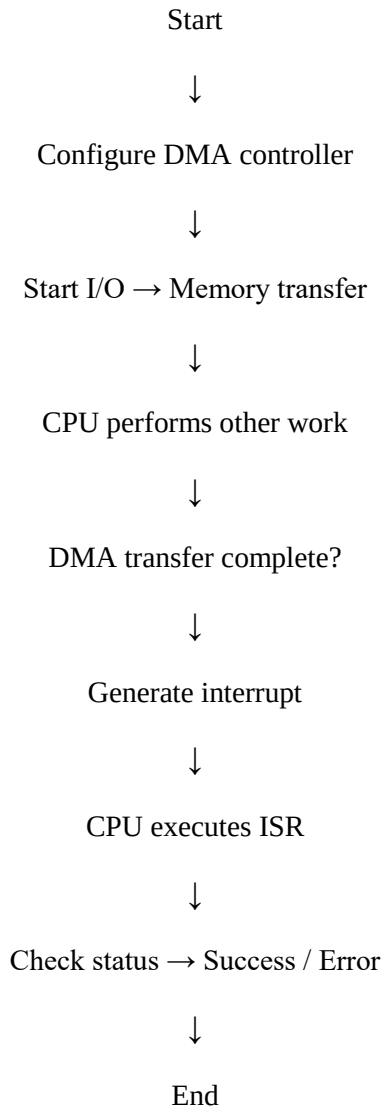
```

ELSE
    Transfer_Status ← "ERROR"
    Handle_DMA_Error()
END IF
END ON

RETURN Transfer_Status
END

```

Flow Diagram



Expected Result

The DMA controller transfers the requested data block directly to memory. The CPU is interrupted only when service is required, thereby reducing CPU overhead and improving I/O efficiency.

Conclusion

The algorithm provides an efficient I/O mechanism because the CPU performs initialization and completion handling while the DMA controller performs the bulk data movement.

2. VECTORED INTERRUPT HANDLING SYSTEM

Question: Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Aim

To design an interrupt handling system that identifies the highest-priority interrupt among multiple devices and dispatches its corresponding interrupt service routine using interrupt vectors.

Algorithm

- Maintain a priority level for every interrupt source.
- Detect pending interrupts.
- Select the highest-priority pending interrupt.
- Use its vector number to locate the appropriate ISR.
- Execute the ISR and clear the interrupt request.
- Return control to the interrupted program.

Pseudocode

```
ALGORITHM VECTORED_INTERRUPT_HANDLER
INPUT: Interrupts from Device_1 ... Device_N
OUTPUT: Serviced interrupt

BEGIN
    Initialize Interrupt_Vector_Table
    Initialize Priority_Table

    LOOP FOREVER
        Pending ← Read_Pending_Interrupts()

        IF Pending is NOT EMPTY THEN
            Highest ← Select_Highest_Priority(Pending)
            Vector ← Get_Interrupt_Vector(Highest)
            ISR_Address ← Interrupt_Vector_Table[Vector]

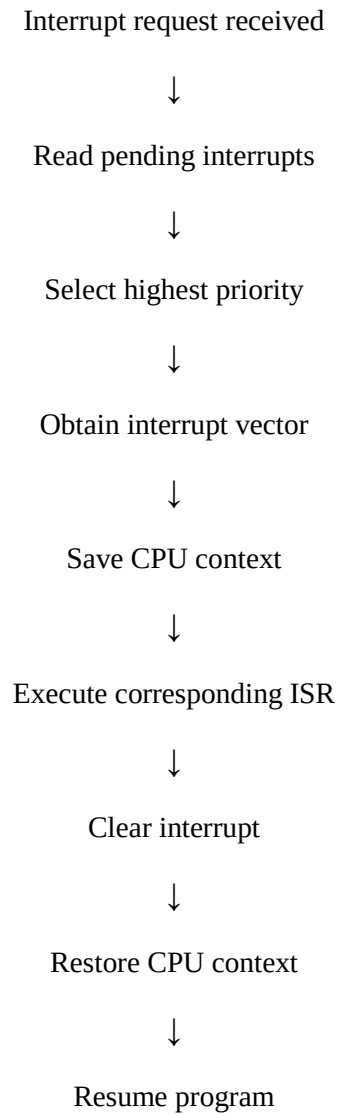
            Save_CPU_Context()
            Acknowledge_Interrupt(Highest)

            CALL ISR_Address

            Clear_Interrupt_Request(Highest)
            Restore_CPU_Context()
        END IF
    END LOOP
```

END

Flow Diagram



Expected Result

The highest-priority interrupt is serviced first and the correct ISR is selected through the interrupt vector table.

Conclusion

Vectored priority interrupt handling provides fast and organized response to multiple devices while maintaining correct CPU context.

3. DYNAMIC SELECTION OF MEMORY-MAPPED I/O AND I/O-MAPPED I/O

Question: Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Aim

To dynamically select an appropriate I/O addressing mechanism according to the application's latency and bandwidth requirements.

Key Idea

- Memory-mapped I/O uses the normal memory address space and can use ordinary load/store instructions.
- I/O-mapped I/O uses a separate I/O address space and dedicated I/O instructions.
- The selection logic evaluates the required latency and bandwidth before choosing a mode.

Pseudocode

```
ALGORITHM SELECT_IO_MAPPING
INPUT: Required_Latency, Required_Bandwidth
OUTPUT: Selected_Mode

BEGIN
    MemoryMapped_Score ← 0
    IOMapped_Score ← 0

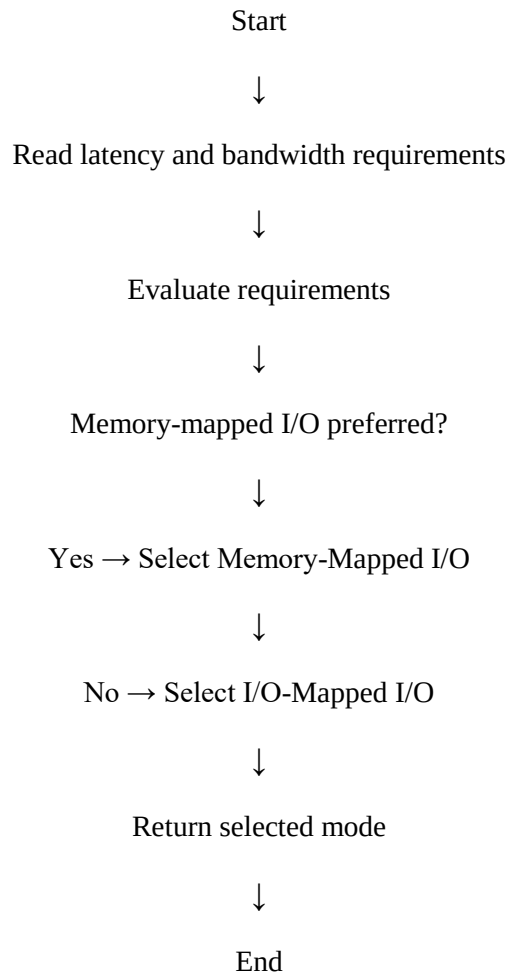
    IF Required_Latency <= LOW_LATENCY_LIMIT THEN
        MemoryMapped_Score ← MemoryMapped_Score + 1
    ELSE
        IOMapped_Score ← IOMapped_Score + 1
    END IF

    IF Required_Bandwidth >= HIGH_BANDWIDTH_LIMIT THEN
        MemoryMapped_Score ← MemoryMapped_Score + 1
    ELSE
        IOMapped_Score ← IOMapped_Score + 1
    END IF

    IF MemoryMapped_Score > IOMapped_Score THEN
        Selected_Mode ← "MEMORY-MAPPED I/O"
    ELSE
        Selected_Mode ← "I/O-MAPPED I/O"
    END IF

    RETURN Selected_Mode
END
```

Flow Diagram



Expected Result

The algorithm selects one of the two I/O mechanisms based on predefined system requirements and decision conditions.

Conclusion

Dynamic selection helps an I/O subsystem choose a suitable addressing approach according to the required performance characteristics.

4. BUS ARBITRATION FOR PCIe, USB AND THUNDERBOLT

Question: Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Aim

To design an arbitration algorithm that manages requests from multiple devices and grants communication access according to priority, interface type, and fairness.

Algorithm

- Collect requests from PCIe, USB, and Thunderbolt devices.
- Check device priority and request urgency.
- Give preference to high-priority and time-sensitive requests.
- Use round-robin selection among equal-priority requests to avoid starvation.
- Grant the interface to one selected requester and update arbitration state.
- Repeat until all pending requests are served.

Pseudocode

```
ALGORITHM BUS_ARBITRATION
INPUT: Request_Queue
OUTPUT: Granted_Device

BEGIN
    WHILE Request_Queue is NOT EMPTY DO
        Requests ← Collect_Active_Requests()

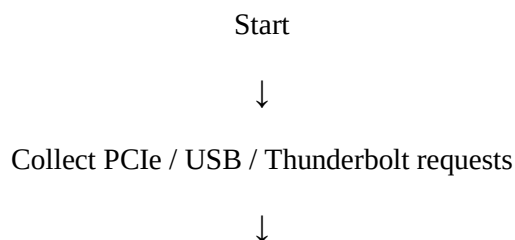
        IF Requests is EMPTY THEN
            WAIT for new request
        ELSE
            Highest_Priority ← Find_Highest_Priority(Requests)

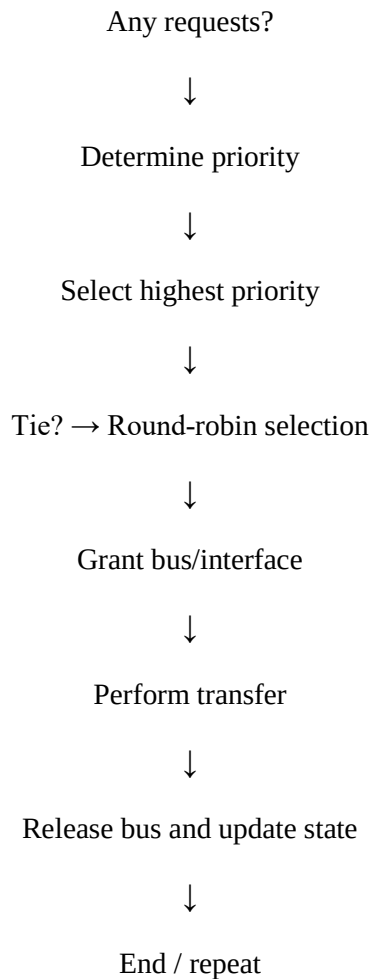
            Candidates ← Requests with
                          Priority = Highest_Priority

            IF Number(Candidates) > 1 THEN
                Granted_Device ← Round_Robin_Select(Candidates)
            ELSE
                Granted_Device ← First(Candidates)
            END IF

            Grant_Bus(Granted_Device)
            Perform_Transfer(Granted_Device)
            Release_Bus(Granted_Device)
            Update_Round_Robin_State(Granted_Device)
        END IF
    END WHILE
END
```

Flow Diagram





Interface Considerations

Interface	Typical Use	Arbitration Consideration
PCIe	High-speed peripheral communication	Prioritize latency-sensitive/high-throughput requests
USB	General peripheral connectivity	Maintain fairness among active devices
Thunderbolt	High-speed external devices	Support high bandwidth and time-sensitive transfers

Expected Result

The arbitration algorithm grants communication access in a controlled and fair manner while giving appropriate priority to urgent or high-performance requests.

Conclusion

Effective bus arbitration prevents conflicts, improves resource utilization, and reduces starvation when several devices request communication resources.

5. HYBRID I/O COMMUNICATION MECHANISM

Question: Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Aim

To design a hybrid I/O mechanism that uses simple polling for low-speed devices and efficient DMA plus interrupts for high-speed devices.

Algorithm

- Identify the device speed and transfer size.
- For low-speed devices, use polling and periodically check device status.
- For high-speed devices, configure DMA and enable interrupts.
- Allow the CPU to perform other work during high-speed DMA transfer.
- Use the interrupt service routine to process completion.

Pseudocode

```
ALGORITHM HYBRID_IO_COMMUNICATION
INPUT: Device, Device_Speed, Data_Size
OUTPUT: Transfer_Status

BEGIN
    IF Device_Speed = LOW OR Data_Size <= SMALL_TRANSFER_LIMIT THEN

        Transfer_Status ← "POLLING"
        Initialize_Device(Device)

        WHILE Device.Data_Ready = FALSE DO
            Check_Device_Status(Device)
        END WHILE

        Read_Data_To_Memory(Device)
        Transfer_Status ← "SUCCESS"

    ELSE

        Transfer_Status ← "DMA_WITH_INTERRUPT"
        Initialize_DMA(Device, Data_Size)
        Enable_DMA_Interrupt

        Start_DMA_Transfer(Device)

        WHILE DMA.Status = BUSY DO
            CPU performs other useful tasks
        END WHILE

        ON DMA_Interrupt DO
            IF DMA.Status = TRANSFER_COMPLETE THEN
```

```

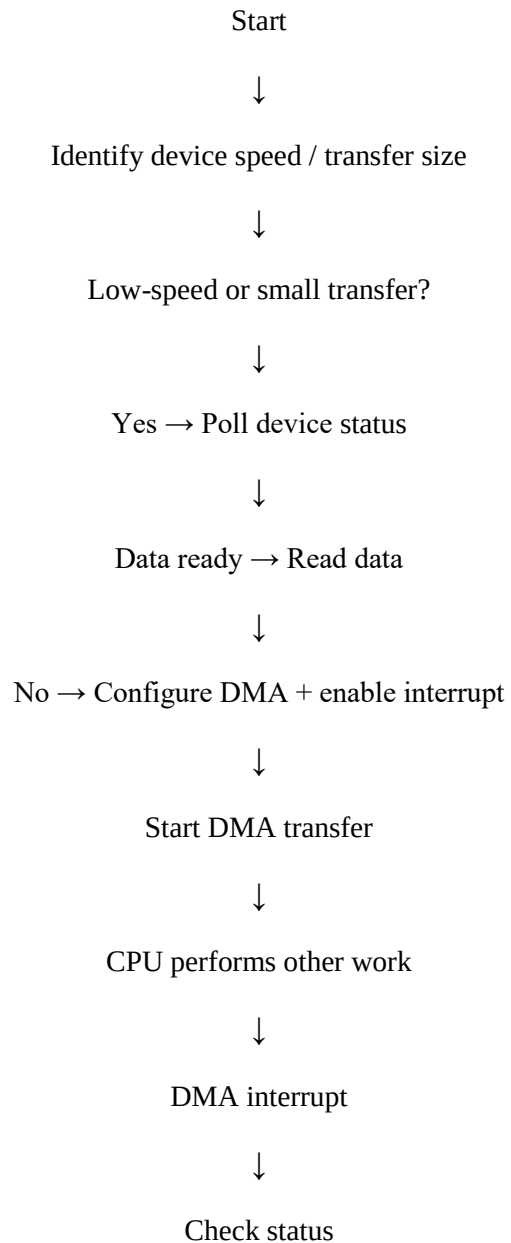
        Transfer_Status ← "SUCCESS"
    ELSE
        Transfer_Status ← "ERROR"
        Handle_DMA_Error()
    END IF
END ON

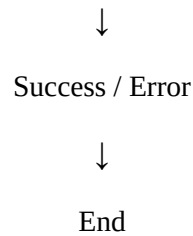
END IF

RETURN Transfer_Status
END

```

Flow Diagram





Comparison of the Two Paths

Feature	Polling Path	DMA + Interrupt Path
Suitable for	Low-speed devices	High-speed devices
CPU involvement	Higher	Lower during data movement
Data movement	CPU reads/writes data	DMA controller moves data
Notification	Repeated status checking	Interrupt on completion
Efficiency	Simple for small transfers	Efficient for large/high-speed transfers

Expected Result

Low-speed devices are handled using polling, while high-speed devices use DMA with interrupts, reducing unnecessary CPU activity and improving overall I/O efficiency.

Conclusion

The hybrid mechanism combines the simplicity of polling with the efficiency of DMA and interrupts, making it suitable for systems containing devices with different performance requirements.

OVERALL CONCLUSION

The five algorithms demonstrate major I/O communication mechanisms covered under CO5. DMA minimizes CPU involvement during bulk data transfers, vectored interrupts provide structured and prioritized interrupt servicing, dynamic I/O mapping selects an addressing mechanism according to requirements, bus arbitration coordinates competing interfaces, and the hybrid mechanism combines polling with DMA and interrupts for devices with different speeds.

RUBRIC ALIGNMENT

The answers are structured to address the assessment rubric provided in the source file: problem understanding, algorithm design, control structures, pseudocode structure, and completeness. The source rubric assigns 20% to problem understanding, 30% to algorithm design, 20% to control structures, 10% to pseudocode structure, and 20% to completeness.

Source basis: The uploaded assessment contains five pseudocode questions on DMA, vectored interrupts, I/O mapping, bus arbitration, and hybrid I/O, followed by the CO5 statement and rubric.

□ filecite□turn0file0□L7-L16□ □ filecite□turn0file0□L28-L31□ □ filecite□turn0file0□L44-L96□